1. Architecture patterns in objected-oriented systems

Architecture patterns are reusable structures that define how the components of a software system are organized and interact.

Two common patterns:

- MVC (Model-view-control)
- Layered Architecture

MVC (Model-view-control) improves modulality by:

- Model - data and business logic
- View - user interface
- Controller - handles user requests.

2. Data Access Object (DAO) pattern:

DAO (Data Access Object) is a design pattern that seperates the database operations from application / business logic.

Advantages

1. Makes database codes easier to maintain
2. Allows the database implementation to be changed easily.

3. Object-oriented design guidelines

Object-oriented design guidelines are principles used to create software that is reusable, maintainable, and flexible.

Two categories:

1. Class-level guidelines -
Focus on designing individual classes

2. Relationship-level guidelines -
Focus on relationships between classes

4. Database persistence and ORM
Database persistence means storing an objects data permanently so that it remains available after the application ends.

Two persistence mechanisms:

1. Object - relational mapping (ORM)

2. Serialization / file storage.

ORM (Object - relational mapping)
ORM frameworks map objects / classes to database tables.

5. Application layer:
The Application layer coordinates application use cases and controls the flow of operations.
Two responsibilities:
1. Coordinate use cases
2. Manage communication between presentation and domain layers.

6. Three - tier - Architecture
A three - tier - Architecture contains

1. Presentation layer
2. Business layer
3. Data layer

1. presentation layer
↳ user interface
2. Business layer
↳ business processing and rules
3. Data layer
↳ database operations.

7. Business Logic Layer
The business logic layer contains and manages the rules and processing of an application.
Two functions:
1. Validate requests
2. process business operations.

8. Cohesion and coupling
Cohesion refers to how closely related the responsibilities of a class / module are
Coupling refers to the dependency between different classes/modules.
Good design aims for:
high cohesion + low coupling

9. Encapsulation and data hiding
Encapsulation means combining data and methods inside a class and controlling access to the data
Datahiding specifically focus on preventing direct access to internal data.

ncy injection
10. Layered online Banking system
A suitable architecture is:
Presentation → Application → Business → Data → Database
Layers:
- Presentation - login and banking interface.
- Application - coordinates transactions.

11. Infrastructure Layer
The infrastructure layer provides technical services required by other application layers.
Two responsibilities:
1. Database access
2. Communication with external systems.

12. Singleton Design pattern:
Singleton is a design pattern that ensures a class has only one instance and provides a common access point to it.
Characteristics:
- Only one object is created
- Provides global
- Controls object creation.

13. Dependency ingection?
→ Dependency injection (DI) is a technique where an object's requred dependancies are provided from outside instead of the object creating them self.
Benefits:
→ Reduce tight coupling
→ Improves testability

14. Strategy Design pattern:

→ The Strategy pattern allows different algorithm / behaviours / to be selected at runtime.

It has 2 main roles:

→ Strategy:
Define the common interface for different algorithm.

→ Context:
Uses the select Strategy without knowing its internal implementation.

15. Library Management System:

Layered architecture:

presentation → application → Bussiness → Data → Database.

Layers:

→ Librarian / student interface

→ Application: coordinates book issue/return operation.

→ Business: Checks availability, borrowing roles

→ Data: performs database operations.